

SISTEMAS OPERATIVOS

Práctica 1

Anthony Eduardo Alvarado

Jean del Pozo Gómez(*jean.delpozo@estudiante.uam.es*)

1. Requisitos cumplidos y no cumplidos

Minería mediante Prueba de Trabajo (PoW): Implementación de un algoritmo de fuerza bruta para encontrar un valor cuyo hash coincida con el objetivo (*target*).

Paralelismo con POSIX Threads: División eficiente del espacio de búsqueda (POW_LIMIT) en rangos disjuntos repartidos entre n hilos.

Comunicación Inter-Procesos (IPC): Uso de tuberías sin nombre (*pipes*) para el intercambio de estructuras de datos entre el proceso Minero y el proceso Registrador.

Sincronización de Procesos: Implementación de un protocolo de confirmación (**ACK**) que garantiza que el minero no avance de ronda hasta que la solución previa haya sido validada y persistida.

El diseño actual se mejoraría si los hilos fueran persistentes. Al sustituir la creación de hilos por ronda por una estructura de hilos reutilizables, se suprimirían los tiempos muertos de gestión del sistema operativo. Esto garantizaría un flujo de trabajo continuo, optimizando el uso de la CPU y evitando el colapso del sistema al incrementar el número de hilos.

2. Pruebas realizadas

Casos de prueba de ejecución

- ☐ Prueba de concurrencia básica: Ejecución con 5 hilos y 5 rondas. Se verificó que los hilos se reparten el rango de búsqueda correctamente y que el primero en hallar la solución detiene a los demás.
- ☐ Prueba de estrés: Ejecución con 1000 hilos. Se comprobó que, aunque el rendimiento cae por el *overhead*, el programa no colapsa y gestiona correctamente la memoria de cada hilo.
- ☐ Prueba de persistencia: Verificación de que el proceso Registrador crea un archivo `miner_[ppid].txt` por cada ejecución y que la información volcada coincide con las trazas de la terminal.

```
==127505== Command: ./mrush 0 5 10
==127505==
Solution accepted : 00000000 --> 08916551
Solution accepted : 08916551 --> 05970796
Solution accepted : 05970796 --> 00937352
Solution accepted : 00937352 --> 01835314
Solution accepted : 01835314 --> 09331340
```

```
Id: [0]
Winner: [127505]
Target: [0]
Solution: 08916551 (validated)
Votes: [0]/[0]
Wallets: [127505]:[0]
```

```
Id: [1]
Winner: [127505]
Target: [8916551]
Solution: 05970796 (validated)
Votes: [1]/[1]
Wallets: [127505]:[1]
```

```
Id: [2]
Winner: [127505]
Target: [5970796]
Solution: 00937352 (validated)
Votes: [2]/[2]
Wallets: [127505]:[2]
```

```
Id: [3]
Winner: [127505]
Target: [937352]
Solution: 01835314 (validated)
Votes: [3]/[3]
Wallets: [127505]:[3]
```

```
Id: [4]
Winner: [127505]
Target: [1835314]
Solution: 09331340 (validated)
Votes: [4]/[4]
Wallets: [127505]:[4]
```

El programa funciona correctamente porque existe una relación entre la terminal y el log: el Target de cada ronda es la Solution de la anterior (ej. el paso de 08916551 de la ronda 0 a la 1), lo que demuestra un encadenamiento lógico perfecto. Todas las soluciones aparecen como validated, confirmando que el proceso registrador verifica y guarda los datos síncronamente antes de que el minero avance. Además, el orden secuencial de los IDs y el PID coincidente (127505) ratifican que la comunicación y el protocolo de confirmación funcionan sin errores.

Casos límite y situaciones de error

Se han testeado escenarios críticos para asegurar que el programa no entre en estados indefinidos:

- ☐ Se probó con un POW_LIMIT muy bajo donde ningún hilo encontraba el hash. El programa sale del bucle correctamente mediante el break en el minero.
- ☐ Se forzó el cierre del proceso registrador prematuramente. El minero detecta que el read del ACK devuelve 0 o error y finaliza la ejecución sin quedar bloqueado eternamente.
- ☐ Se introdujo un target inicial que ya es una solución. El sistema procesa la ronda 0 instantáneamente y avanza a la siguiente sin errores de lógica.
- ☐ Se probó con un número de hilos que no divide exactamente a POW_LIMIT (ej. 3 hilos). Se verificó que el reparto de rangos mediante count += range_size cubre la mayor parte del espectro sin solapamientos.

4. Respuesta a las cuestiones del enunciado

Tiempos de ejecución

Num_hilos\Rondas	5	10
1	2,18s	3,93s
5	1,63s	3,44s
10	1,71s	4,08s
100	2,92s	5,33s
1000	6.17s	11,21s

El rendimiento óptimo se alcanza con 5 hilos, que es el punto donde mejor se aprovechan los núcleos de la CPU sin estorbarse entre sí. A partir de los 10 hilos, el tiempo empieza a subir porque el sistema operativo gasta más energía en saltar de un hilo a otro (cambio de contexto) que en minar realmente, lo que demuestra que utilizar más hilos no siempre significa más velocidad.

En casos exagerados como los 100 o 1000 hilos, el tiempo de ejecución se dispara (hasta triplicarse) porque el ordenador se colapsa gestionando la tarea de crear, vigilar y cerrar tantos hilos en cada ronda. Al final, la conclusión es que el paralelismo funciona de correctamente hasta que se llega al límite físico del procesador dado que a partir de ahí, el exceso de gestión se vuelve un lastre que frena todo el proceso.

El rendimiento mejoraría drásticamente si los hilos se creasen una sola vez al inicio del programa. En el diseño actual, el sistema pierde tiempo al encargarse de abrir y cerrar hilos en cada ronda. Si se mantuvieran los hilos vivos (Thread Pool) y solo les diera el nuevo target al empezar cada ronda, se eliminaría ese tiempo muerto de creación y destrucción, logrando que el paso de una ronda a otra fuera casi instantáneo y que el sistema escalara mucho mejor sin colapsar la CPU.